Table of Contents

Private Texas Hold'em — Software Design Document (SDD)

Version: 1.0 · **Status:** Baseline design for implementation · **Date:** 2026-10-10
Audience: Product owner, full-stack developers, QA engineers and coding agents
Scope: Browser-based invitation-only No-Limit Texas Hold'em Cash Game with virtual integer chips, 2–9 seated players per table and 10–30 users online in total.

> **Product boundary.** The app must not accept deposits, initiate payments, calculate transfers between people, provide withdrawals or automate settlements of real-world money. Players may independently arrange matters outside the product; the system only records virtual chip totals. Because outside settlements may involve gambling laws, jurisdiction-specific legal review is required before operating for stakes. This document is not a legal opinion.

1. Executive architecture decision

Build a **modular monolith**: Next.js/TypeScript/Tailwind client; FastAPI/Python server; managed PostgreSQL; authenticated secure WebSocket transport; a single always-on backend instance and durable DB-backed room/hand states. No Redis, Kafka, microservices or Kubernetes for v1. A pure domain poker engine must be callable without HTTP/WebSocket. Server state is authoritative. Mutations are serialized by room and committed atomically before any success broadcast.

1.1 Architecture

Desktop / Mobile Browsers
└─ Next.js App: lobby, table, chips, chat, replay
  ├─ HTTPS REST → FastAPI API (sessions, rooms, history, admin)
  └─ WSS WebSocket → FastAPI Realtime Gateway
    ├─ Authentication and access policy
    ├─ Per-room action coordinator and timer
    ├─ Pure Poker Engine / Hand Evaluator
    ├─ Room, Seat, Host and Session services
    ├─ Chip Ledger / Hand Event store
    └─ Authorized projection + event dispatcher
      ↕️
    Managed PostgreSQL

1.2 Invariants

- The backend shuffles, deals, verifies actions, selects acting player, evaluates hands and allocates pots. Clients never submit cards, chip balances or winner determinations.

- For any poker action, action_id is unique per actor/room; repeated delivery must return the original result without applying changes twice.
- One durable transition consumes one version and produces the next room_version, with a transactionally appended event; connected clients receive updates only after commit.
- Each room has its own balance universe; chips are never transferred between rooms.
- For a complete hand, total participant stacks plus all outstanding player commitments/pots are conserved, excluding explicit between-hand buy-in/top-up/cash-out operations.
- Never expose another player's private hole cards, deck seed/deck remainder, unrevealed mucked cards or private sessions in general snapshots, history or spectator events.
- Money uses **integer chip units**, BIGINT with CHECK >= 0 where applicable; amounts and positions are not JavaScript floating point calculations.

2. Approved product requirements

Area	Confirmed behavior
Entry	Anonymous guest nickname; persistent secure session token; nickname is display-only, not identity
Rooms	Multiple private rooms; invite URL and password required; room-scoped ban list; host controls
Seats	User chooses empty seat 1–9; spectators allowed; at least 2 eligible players to start a hand
Game	No-Limit Hold'em Cash Game, room-configured small/big blinds and min/max buy-in
Hand start	Host starts **each** hand; once started streets advance automatically
Buy-in	Room-local virtual chips; min/max per room; host accepts requested top-ups, applied between hands
Blinds entry	Wait for Big Blind or Post Big Blind with a defined missed-blind ledger
Actions	Fold, Check, Call, Bet, Raise, All-in; actionable 30 s server timer
Timeout	Check if legal; otherwise Fold; all-in player has no further decisions
Exit	Leave during live hand folds if a legal fold is possible; all-in remains until settlement; cash-out afterward

Area	Confirmed behavior
Sit out	Retain stack and seat, excluded from new hands; blind reentry policy applies when returning
Host	May kick/ban, approve top-ups, update future blinds, start hands, transfer host
Host disconnect	60 s grace then oldest eligible connected room member becomes host; former host retains guest membership
Restart	Restore last committed state; pause action timer; active host resumes play manually
History	Room members may review action-by-action public history, chip transactions, outcomes and replay
Spectate	Public board, bets, positions and public revealed hands only; no other hidden data
Chat	Room text chat with rate limits and sanitization
Closure	Empty room closes after 24 h; hand/ledger records kept as historical archives
UI	Responsive desktop and mobile, modern dark table, bet slider, ½-pot/pot/all-in presets, replay

3. Roles, identity and authorization

Guest identity. guest_id is a random opaque UUID. On first entry create a DB-backed server session. Send only a session identifier using an HttpOnly; Secure; SameSite=Lax cookie with a bounded lifetime and rotation on sensitive transitions. Never identify accounts solely by nickname, IP address or localStorage. Joining invite links requires the cookie plus room password. A password is stored via Argon2id or comparable strong password hash, never plaintext; rate-limit guesses. Invite URL contains unguessable room invitation token, not the password. Password is still required.

Roles. host: normal player privileges plus moderation/configuration/resume/start/approve. member: choose seat, play, sit out, top-up request, read room history, chat. spectator: watch public game and chat (if room membership approved) but not play or see hidden cards. One guest may be a member and spectator at the same time in the room, but a seat uniquely belongs to one guest.

Ban model. Host can ban a stable guest_id for the room and may unban; changing browsers can evade bans, unavoidable with guest-only identities. On kick, deny new commands immediately; if seated in an active hand, mark pending removal, fold only if still entitled to act and not already all-in, settle afterward, then remove seat and record cash-out. Do not confiscate

chips. The replaced host inherits permission to edit previous bans. No host may arbitrarily alter another player's in-hand stack.

Host transfer. After 60 s continuously disconnected, elect the earliest-joined eligible connected non-banned room member, using a deterministic sort of joined_at, member_id. Never allow two simultaneous hosts: transactionally update host_member_id with a room version guard. If no eligible player connected, retain vacancy and elect on next eligible join. On recovery after restart, timer remains paused; present host at that time may Resume. Host who reconnects after transfer does not automatically reclaim authority.

4. Room and seat lifecycles

Room: OPEN → EMPTY_PENDING_CLOSE (no connected members) → CLOSED after 24 h; returning member within 24 h returns room to OPEN. Closure must not erase hands or chip history. A live hand with no connected members must not be auto-deleted or paid out; remain recoverably paused. The 24 h closure job must first resolve or flag unresolved hands for admin/recovery, never silently discard a live pot.

Seat: EMPTY, WAITING_FOR_BLIND, ACTIVE, SITTING_OUT, DISCONNECTED, LEAVING_PENDING, CASHED_OUT. These are conceptual states; operationally store participation mode separately from connectivity and pending departure to avoid conflicting flags. A new member joining in the middle of a hand selects an available seat but cannot enter that already started hand.

Entry and missed blinds: A new/rejoining player chooses (a) wait for eligible BB position or (b) post allowed entry blind before the next eligible hand. Determine missed-blind debt consistently, including dead small-blind cases. Never insert a player mid-hand; heads-up uses its own dealer/SB rules. v1 requires rule-specific acceptance tests before activating post-BB option. **Default detailed policy proposed for implementation:** Use a traditional missed-blind algorithm with live BB plus dead SB where owed; if details cannot be validated against a reference rule set, disable manual posting and allow Wait for BB only until validated. This retains the committed "standard" entry approach without shipping flawed blind logic.

Leave and sit out: SIT_OUT opts out starting the next hand, or if currently active, becomes effective after the current hand; to leave immediately mid-hand, auto-fold if it is legal and mark departure; all-in waits for the end. A seated player is never automatically cashed out solely because connectivity is lost. Sitting out does not invoke auto-rebuy. Host may kick a disconnected seat following the same pending-settlement rules.

Configuration updates: Host may change blinds while a hand is live, but store pending_sb / pending_bb; changes apply at **next Start Hand**. Each hand copies a frozen config snapshot (blinds, seat order, buy-in bounds and evaluator/rules version). Validate positive integers and SB < BB; disallow ambiguous retroactive changes. If buy-in bounds are adjusted, the current hand is unaffected and existing stacks remain valid.

5. Poker domain engine specification

5.1 Phases and transitions

WAITING_FOR_HOST → HAND_SETUP (seating, blind positions, deck shuffle, forced bets, deal) → PREFLOP → FLOP → TURN → RIVER → SHOWDOWN → SETTLEMENT → HAND_COMPLETE → WAITING_FOR_HOST.

Short-circuits: all but one fold ⇒ immediate uncontested payout; everybody remaining all-in ⇒ run out board through river then resolve pots; no more possible betting ⇒ automatically advance street. PAUSED_RECOVERY is an operational overlay rather than a betting street. The hand state must indicate the exact next actor, street commitments, last full raise size, who can still raise and action deadline.

5.2 Dealing, turn order and validation

Use cryptographically strong randomness from OS CSPRNG for Fisher–Yates shuffle, never `random.Random` for real games. Deck and hidden card state are persisted encrypted at rest; shuffle exactly once per hand, immutable thereafter. In 3+ player hands, preflop first action follows the BB (under the gun); postflop first action is earliest eligible seat left of button. With 2 players, button is SB and acts first preflop, BB acts first postflop. Automatic blinds are limited by available stack and may produce all-in.

Action command includes `hand_id`, `action_id`, `expected_room_version`, action type and `raise_to` (the desired total **street** contribution, not increment). Validate membership, seat, acting turn, deadline, active street, available stack and minimum/legal action. Handle opening bet, call, short call, full raise, short all-in, and whether a non-full raise reopens action. Calculate available actions and min/max legal raise on server and emit them only for the acting player's private view. Clients display but never decide eligibility.

30 s timer starts on commit of the previous action/state transition and stores an absolute deadline in DB. Race between timeout and player action: whichever first wins serial mutation for that room decides; post-deadline actions must be rejected based on server clock. Timeout chooses Check if legal, else Fold. Turn timeout and disconnect processing must not invoke duplicate player actions.

5.3 Side pots and showdown

Maintain per-player cumulative contribution across all streets and per-street contributions separately. At settlement, segment total contributions into contribution thresholds; each pot has an amount and a set of eligible non-folded contenders. Folded contributions fund pots but are never eligible to win. Apply uncalled-bet refunds before contested distribution. Evaluate the best five-card combination of each eligible 7-card hand. Split ties as integers; allocate remainder chips by a deterministic documented seat order starting clockwise from the button among winners. Show the minimum information required to establish winning claims under house rules; allow eligible losing hands to Muck when not required to reveal; all-in showdown disclosures must follow a documented reference rule set. Never leak unrevealed hands in replay.

Evaluator quality gate: Test straight/flush edge cases, wheel A-2-3-4-5, exact 5-card selection, kickers, full house pair ordering, tied boards, side pots, odd chips, short all-in and heads-up. Use a trusted independent evaluator as an oracle in property tests where possible.

5.4 Chips and settlement

Buy-in, approved Top-up, Cash-out are journaled as immutable room-chip transactions, separate from within-hand chip movements. Approval is distinct from application; host acceptance queues a top-up to be applied atomically between hands, only if resulting stack is within configured max buy-in. No top-up affects a live hand. Each action updates materialized stacks, street contributions and immutable action event atomically. Net chip result per room session is cash_out + current_stack - total_buyins - total_topups (adjust for any explicit logged authorized correction; standard flow has none). An entry/exit reconciliation report is required; wallet or real-money ledger is out of scope.

6. Data model and constraints

Table	Main fields	Guarantees
guests	guest_id PK, nickname, created_at	Nickname non-unique, never auth key
guest_sessions	id PK, guest_id FK, token_hash, expires_at, revoked_at	Indexed token hash, secure expiry
rooms	id PK, invite_token_hash, password_hash, host_member_id, status, config_json, pending_config_json, version, empty_since	Version monotonic, host FK deferred as needed
room_members	id PK, room_id FK, guest_id FK, joined_at, role, ban_at	Unique (room_id, guest_id), except deliberately modeled rejoin sessions
room_bans	room_id, guest_id, banned_by, created_at, revoked_at	At most one active ban per guest+room
table_seats	room_id, seat_no, member_id, stack, participation_state, pending_leave	Unique (room_id, seat_no), member seated once per room, stack >= 0
hands	id PK, room_id, ordinal, phase, button_seat, actor_seat, config_snapshot, room_version, deadline_at, hand_snapshot_encrypted,	Unique (room_id, ordinal), immutable rules snapshot

Table	Main fields	Guarantees
	started_at, ended_at	
hand_players	hand_id, member_id, seat_no, hole_cards_ciphertext, street_contribution, total_contribution, folded, all_in, show_status	Never exposed unredacted
hand_events	id PK, hand_id FK, seq, action_id, actor_id, event_type, public_payload, private_payload_ciphertext, version, created_at	Unique hand+seq and actor+action_id; append-only
chip_ledger	id PK, room_id, member_id, type, amount, reference_id, created_at, applied_at	Immutable, unique application idempotency reference
topup_requests	id PK, room_id, member_id, amount, status, approved_by, applied_at	REQUESTED→APPROVED→APPLIED or rejected
chat_messages	id PK, room_id, member_id, body, created_at	Length limits, escaping, retention policy

Additional indexes: (room_id, status), (room_id, joined_at), (hand_id, seq), (member_id, created_at) and pending timers. Use encrypted snapshot fields or separately encrypted secrets; never log hole cards, session cookies or invite secret in application logs.

6.1 Example durable room snapshot


```
{
 "room_id": "room_uuid",
 "room_version": 108,
 "hand_id": "hand_uuid",
 "phase": "TURN",
 "operational_status": "ACTIVE",
 "button_seat": 4,
 "actor_seat": 7,
 "current_bet": 40,
 "min_full_raise_increment": 20,
 "deadline_at": "2026-10-10T22:30:00Z",
 "seats": [{"seat_no": 7, "stack": 160, "street_contribution": 20}],
 "community_cards": ["Ah", "7c", "7d", "2s"],
 "private_deck_ref": "encrypted-internal-storage-only"
}
```

Example above is illustrative, not exhaustive and must never be sent unchanged to clients.

7. REST API contract v1

Prefix /api/v1; JSON, timestamps ISO 8601 UTC, ids UUID. Enforce cookie session and CSRF measures on state-changing REST requests; never authorize based solely on client-reported guest IDs.

Method / path	Purpose	Key request → response
POST /guests/session	Create/resume guest	{nickname} → safe guest profile; Set-Cookie
GET /me	Current guest and memberships	→ current guest + room summaries
POST /rooms	Create password room	{password, sb, bb, min_buyin, max_buyin} → room + invite URL
POST /rooms/{id}/join	Verify invite token and password	{password, invite_token} → member
GET /rooms/{id}	Safe lobby snapshot	→ role-dependent room public state
POST /rooms/{id}/seats/{n}	Claim seat and buy-in	{buyin} → seated state
POST /rooms/{id}/sit-out	Set participation preference	{sit_out: true/false} → acknowledged
POST /rooms/{id}/leave	Leave/cash-out workflow	→ pending or completed state
POST /rooms/{id}/topups	Request top-up	{amount} → request id
POST /rooms/{id}/topups/{requestId}/approve	Host decision	{approve} → status
POST /rooms/{id}/start-hand	Host starts next hand	→ hand id and version
POST /rooms/{id}/resume	Host resumes recovered hand	→ hand state and deadline
PATCH /rooms/{id}/config	Update next-hand blinds/bounds	{sb,bb,...} → pending config
POST /rooms/{id}/members/{memberId}/ban	Host kicks/bans member	→ ban and pending departure
DELETE /rooms/{id}/bans/{memberId}	Host lifts ban	→ state
POST /rooms/{id}/transfer-host	Host delegates	{member_id} → host

Method / path	Purpose	Key request → response
GET /rooms/{id}/hands?cursor=	Paginated safe history	→ historical hand summaries
GET /rooms/{id}/hands/{handId}	Redacted public replay record	→ ordered public hand events
GET /rooms/{id}/ledger?cursor=	Room-member ledger visibility	→ member-specific or authorized room ledger per policy

POST /rooms/{id}/start-hand must reject with documented errors unless at least two eligible seats, no active unresolved hand, host authorized and correct room version. Repeated start requests with same idempotency key must not start two hands. Prefer WebSocket for in-game actions; REST is not a fallback that bypasses serialization.

7.1 Error envelope

```
{
  "error": {
    "code": "NOT_YOUR_TURN",
    "message": "Only the current actor may submit a poker action.",
    "room_version": 108,
    "request_id": "request_uuid"
  }
}
```

Canonical error codes include UNAUTHORIZED, ROOM_BANNED, ROOM_CLOSED, ROOM_FULL, INVALID_PASSWORD, INVALID_BUYIN, NOT_HOST, HAND_ALREADY_RUNNING, NOT_YOUR_TURN, INVALID_ACTION, STALE_VERSION, DEADLINE_EXPIRED, TOPUP_LIMIT_EXCEEDED and RATE_LIMITED. Never leak internal stack traces.

8. WebSocket protocol and consistency

Connection: wss://.../ws/rooms/{room_id} with browser cookie and Origin check. Revalidate membership and ban status on connect and each privileged command. Client sends no session token in query strings. Heartbeat/ping-pong detects disconnections; server—not client—decides connected status.

Command envelope:

```
{
  "kind": "command",
  "type": "player.action",
  "action_id": "b3f...uuid",
  "hand_id": "hand_uuid",
  "expected_room_version": 108,
```

```
  "payload": { "action": "RAISE", "raise_to": 50 }
}
```

Server event envelope:

```
{
  "kind": "event",
  "type": "game.state",
  "event_id": "uuid",
  "room_id": "room_uuid",
  "room_version": 109,
  "server_time": "2026-10-10T22:29:31Z",
  "payload": { "phase": "TURN", "pot": 110, "actor_seat": 3 }
}
```

Events: room.state, room.host_changed, player.joined, player.left, player.turn, hand.started, hand.street_changed, hand.completed, game.state, action.result, chip.requested, chip.approved, chip.applied, chat.message, room.paused, room.resumed, room.closed. Public event payloads are generated from per-recipient redacted projections. Private hole card notifications go only to the owning guest's session(s) over a separate directed event (hand.your_cards). A replay connection must have the same access filtering.

Ordering and catch-up: Each successful state mutation increments a room version. WebSocket events may be lost or arrive late, so clients compare versions and fetch a fresh authorized snapshot on gap, reconnection or mismatch; do not apply out-of-order state blindly. The server stores action deduplication results so retries return original outcomes even after restart. One room's commands are serialized under a logical room mutex; a DB row lock/version check is still required to protect against other processes/tasks. For 10–30 users, a single process with a single worker and dedicated coordinator is preferred; do not configure multiple Uvicorn workers without distributed coordination.

Concurrency transaction outline: acquire per-room logical lock → load room with SELECT ... FOR UPDATE → reject stale/late command → run pure engine → write snapshot, hand event and chip-related deltas → commit → build authorized projections → publish. If process crashes after commit but before publish, a client snapshot/reconnect must reconstruct truth. Optional DB outbox can be added if stronger delivery is needed later.

9. Crash recovery, host authority and timers

Every committed transition includes enough private durable data to reconstruct the exact hand, deck remainder, action order, all contributions, pending departure flags, and timeout actor. After process restart, inspect active rooms and set operational_status=PAUSED_RECOVERY. Do not execute overdue actions during recovery. Notify reconnected clients of frozen state. Require currently authorized host to click Resume. Recreate the current actor's 30-second deadline **from resume time**, commit new version, then announce play. If original host is absent after configured 60-second disconnected grace, elect another eligible connected member; new host can Resume. If no one can resume, keep

recoverable state until expiry/reconciliation policy is invoked by an authorized operator—never silently forfeit players' stakes/chips.

Timer scheduling is in process, but the authoritative deadline lives in PostgreSQL. On each wake-up, reload expected hand id/version and validate deadline before applying automated Check/Fold. Host transfer and room-empty closure similarly use persisted timestamps and idempotent jobs.

10. History, replay and redaction

Write every accepted gameplay command, forced bet, street advancement, board reveal, win split and relevant moderation action with monotonic hand sequence. Historical replay shows the publicly observable actions and cards as of that historical point; it must not expose cards learned later prematurely, nor hidden mucked hands at any time. If a player's cards are optionally shown at showdown, only reveal from the associated show event forward. Recorded timestamps are not a substitute for total-order sequence numbers. The room's past members with valid guest identity may access closed-room archives; banned members can access their own chip receipts and permitted historical records subject to explicit room access policy, but cannot rejoin live play.

Retention: Keep hands/chip ledger as historical records until a documented archival/deletion policy is enacted. Purge expired sessions separately. Treat chat retention and privacy deletion separately from financial-like audit logs, with backups aligned to retention. Closed rooms are archived, not physically deleted by 24 h closure job.

11. Frontend UX blueprint

Lobby: create room wizard (name optional, password, SB/BB, buy-in bounds), invite link copy, join password input, available room state, guest nickname setting.

Poker table desktop: modern dark felt, 2–9 seats, chips stacks, seat badges, dealer/SB/BB indicators, community cards, pot/main+side pot labels, 30-second clock, action row, Raise slider with server-derived legal min/max and quick buttons, private hole cards anchored to local player only. Visible queued top-ups and host prompts should not obscure acting buttons.

Mobile: portrait first, tap targets at least ~44 px, sticky action bar, safe-area insets, cards and chips readable without zoom, chat/history as drawers, stable layout around notches/virtual keyboards. Hide private cards on app background only as UX mitigation, not security boundary. Avoid reliance on animation for gameplay; optional motion reduction. Audio and elaborate animations are excluded by final selection.

Spectator: only public cards and bets, no actionable controls; may leave/claim vacant seat and enter at a legal hand boundary. **Replay:** sequence timeline with Previous/Next/Play; controlled by logged events, showing delayed/mucked cards only if public at that step.

12. Operational, security and performance design

- Managed provider must support always-on WebSocket connections and an always-on single backend (no cold-sleep), managed Postgres, encrypted network transport, backups and health checks. Candidate cloud vendors are not mandated.

- Configure TLS/HTTPS/WSS, strict allowed Origins, CSRF protection for cookie-auth REST, secret storage, Argon2id password hashing, IP/session rate limits, payload size limits, sanitization and secure headers.
- Enforce server authorization at every REST, WS command and state projection; spectators and banned users must not access hidden-card events. Encrypt private card/deck data at rest; log only redacted structured identifiers.
- Service metrics: active rooms/connections, action latency p50/p95, WS disconnect/reconnect rate, timeout count, version-conflict rate, hand settlement errors, chip conservation invariant failures, DB transaction latency.
- Health/readiness: /health/live, /health/ready; readiness should include DB connectivity and recovery state. On deploy/rolling restart, stop accepting new hand starts, commit snapshots, close/reconnect WebSockets and perform pause/resume flow.
- Default resource guidance: one small managed always-on app instance and entry-level managed Postgres, then load-test. Do not guarantee specific throughput without measurements.
- Time sync: server UTC authoritative for deadlines; client may estimate countdown using transmitted server time but does not decide timeout.
- Backup/recovery must be periodically tested by restoring a copy and verifying ongoing hand snapshots and chip ledger invariants.

13. Test strategy and acceptance criteria

13.1 Unit/property tests (Poker Engine)

- 2-player button/SB ordering; 3–9 players; seat vacancies; automatic forced bets and debt on reentry.
- All basic player actions, rejected action out of turn, insufficient stacks, min/full/short raises, all-in, non-reopening short raise.
- All street transitions; early uncontested win; 0/1/2+ side pots; ties, odd chips, uncalled bet refunds.
- 7-card evaluator including wheel, straight flush, board plays, kickers and equal ranks.
- Conservation/property tests across generated legal sequences: no negative chips, no duplicated cards, max 52 unique cards, contribution balances correct.

13.2 Integration/realtime tests

- Duplicate action ID; simultaneous action vs deadline; stale expected version; crash between commit and broadcast; crash while active hand.
- Guest reconnect same seat; two tabs same guest; host disconnect transfer after 60 s; transfer race; recovered host Resume.
- Password guessing rate-limit; wrong invite link; banned session reconnect; spectators cannot see hole cards; history redacts mucked cards; no cross-room data.
- Top-up approval during a live hand applied only at the next boundary; cap enforcement; leave during hand and leave while all-in; room closes after 24 h empty while archives remain.
- Full 30 concurrent-user scripted play, latency measurement, phone viewport tests via Playwright.

13.3 Release gates

Gate	Criteria
Core poker	Repeatable engine tests covering all key Hold'em edge cases
Consistency	No negative stack or conservation mismatch across simulation and concurrency tests
Recovery	Restore hand to last committed state; host Resume without duplicate card/action/chips
Privacy	No WS/REST/history/replay leak of unrevealed opponent hole cards
Authorization	Guest/session, host privileges, ban and room boundaries enforced server-side
Mobile	All essential actions usable on portrait mobile and desktop
Capacity	30 simulated simultaneous users without game-state divergence

14. Repository and module boundaries

```
poker-platform/
  apps/
    web/              # Next.js client, UI, types, WebSocket hooks
  services/
    api/
      app/
        main.py
        api/          # REST controllers, auth dependencies
        realtime/     # WebSocket transport + projections
        domain/poker/ # PURE engine, state machine, evaluator
        services/     # rooms, members, host, chip ledger, timers
        persistence/  # SQLAlchemy models, unit-of-work, repos
        security/     # session and policies
      migrations/     # Alembic migrations
      tests/unit/
      tests/integration/
      tests/property/
  infra/              # Container build, IaC/deployment configs
  docs/               # API/WS schemas, ADR, runbooks
```

Dependency direction: transport → application services → pure domain; persistence implements interfaces used by services. Poker domain must not import FastAPI, SQLAlchemy, WebSocket or UI modules. All DTOs/schema versions are pinned; TypeScript client types should be generated from OpenAPI and a shared documented WS event schema if practical.

15. Implementation plan

Phase	Deliverable	Exit test
0. Foundation	Repo, CI, Docker dev setup, Postgres migrations, secure sessions	Integration smoke test
1. Rules engine	State machine, shuffle, evaluator, betting, pots	Extensive deterministic/property tests
2. Durable multiplayer	Room/seat/host, DB snapshots, ledger, actions/timers, WS	Simulated full hands, concurrency tests
3. Player experience	Desktop/mobile poker table, lobby, invitation/password, chat	Real-device usable flows
4. Admin + archive	Host controls, top-ups, bans, replay/history, sit-out/leave	Role and redaction test suites
5. Operations	Recovery/Resume, observability, backup tests, cloud deployment	Restart and 30-user load gates

16. Open implementation decisions / assumptions register

The six rounds settle all top-level product features, but the following values must be made explicit in engineering tickets before coding affected flows. Proposed defaults are not user-confirmed product requirements unless indicated:

ID	Item	Proposed decision / constraint
A-01	Post-BB/missed-blind exact rule set	Reference a named house/tournament rulebook and write examples; disable nonvalidated path until tested
A-02	Show/Muck/all-in disclosure	Specify hand-level required reveal ordering from selected reference rules; preserve mucked secrecy
A-03	Access after room closure/ban	Prior members see nonsecret history; banned users see own ledger without live room access
A-04	Buy-in cap after blinds change	Max total sitting stack applies to approved top-up, existing larger stacks grandfathered

ID	Item	Proposed decision / constraint
A-05	Host unavailable during recovery	Transfer based on 60 s continuous disconnect to earliest connected eligible member
A-06	Deck encryption/key rotation	App-managed envelope secret via cloud secrets manager, backup key recovery playbook
A-07	Session longevity	Default 30-day sliding expiration with token rotation; user may reset guest session; adjustable
A-08	Replay/retention policy	Keep hand+ledger records; independent bounded chat/session retention and privacy requests
A-09	Nonintegral pot presets	Round preset amounts to legal integer street total and clamp by server bounds; server always validates
A-10	Full game recovery if no eligible host	Hold paused indefinitely until an authorized operator resolves; alert before any data archival

Definition of design sign-off: Product owner accepts the above assumptions or asks for targeted changes; poker-rules acceptance tests define normative behavior. This SDD is a baseline architecture document, not proof that code is implemented or behavior has been verified.

Appendix A. Example authoritative action pseudocode

```
async def apply_player_action(room_id, session, command):
  async with room_mutex(room_id):
    async with db.transaction() as tx:
      room = await tx.lock_room(room_id)
      actor = authorize_seated_member(room, session)
      prior = await tx.find_action_result(room_id, actor.id, command.action_id)
      if prior is not None:
        return prior  # idempotent retry
      require(room.version == command.expected_room_version, "STALE_VERSION")
      require(not room.paused, "ROOM_PAUSED")
      require_server_deadline_not_elapsed(room)
```

```
            next_state, events = poker_engine.reduce(room.state, actor, command)
            poker_engine.assert_invariants(next_state)
            await tx.save_snapshot(next_state, version=room.version + 1)
            await tx.append_events(events)
            await tx.save_action_result(command.action_id, result="accepted")
        await websocket_gateway.publish_redacted_room_snapshot(room_id)
    return {"accepted": True}
```

Appendix B. Source-of-truth conventions

- rooms.version / hands.room_version are monotonic committed server state versions, not UI clocks.
- action_id protects retries; event_id identifies durable publication; hand_events.seq is replay order.
- A raise_to amount is always a **total contribution in the active betting street**.
- Timeouts, host transfers and cleanup jobs carry explicit idempotency keys and version checks.
- A room archive is **not** deletion; an empty-room scheduled close must not erase unsettled state.